

Reviewer Pack — Matroid Rank Submodularity and Monotone Circuit Size for k-C...

Entry b052cc185466 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 02:20:52 UTC

Matroid Rank Submodularity and Monotone Circuit Size for k-CLIQUE

Entry ID: b052cc185466 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 02:20:52 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Monotone Circuit Size for k-CLIQUE

Statement:

For a k-CLIQUE CNF formula Φ on n variables, the rank function $r(S)$ of the matroid M constructed from Φ satisfies $r(S) \geq \Omega(n^{\{1/2\}})$ for any subset S of size $\Omega(n^{\{1/2\}})$, and the minimum number of elements needed to achieve $r(S) = \Omega(n^{\{1/2\}})$ is $\Omega(n^{\{1/2\}})$.

Rationale (proposer's reasoning):

The submodularity of the rank function in matroid theory could mirror the structure of monotone circuits, where dependencies between clauses (elements) affect the circuit's size. By linking the rank's growth rate to the circuit's complexity, we might uncover structural constraints.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0351cbab391e2c94

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=-1, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_prime(n):
        if n <= 1:
            return False
        for i in range(2, int(math.sqrt(n)) + 1):
            if n % i == 0:
                return False
        return True

    def generate_primes(count):
        primes = []
        num = 2
        while len(primes) < count:
            if is_prime(num):
                primes.append(num)
            num += 1
        return primes

    def gaussian_elimination(matrix):
        m, n = len(matrix), len(matrix[0])
        for i in range(m):
            max_row = i
            for j in range(i + 1, m):
                if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                    max_row = j
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
            if matrix[i][i] == 0:
                continue
            for j in range(n):
                matrix[i][j] /= matrix[i][i]
            for j in range(m):
                if i != j:
                    factor = matrix[j][i]
                    for k in range(n):
```

```

        matrix[j][k] -= factor * matrix[i][k]
    return matrix

def rank(matrix):
    m, n = len(matrix), len(matrix[0])
    row echelon_form = gaussian_elimination(matrix)
    rank = 0
    for i in range(m):
        if any(echelon_form[i][j] != 0 for j in range(n)):
            rank += 1
    return rank

def generate_k_clique_cnf(n, k):
    clauses = []
    variables = list(range(1, n + 1))
    for i in range(k):
        clause = random.sample(variables, 2)
        clause.append(-random.choice(clause))
        clauses.append(clause)
    return clauses

def matroid_rank_function(cnf, S):
    matrix = [[0] * (n + 1) for _ in range(n + 1)]
    for literal in cnf:
        if all(abs(literal[i]) not in S for i in range(2)):
            continue
        for i in range(2):
            var = abs(literal[i])
            matrix[var][var] += 1
            for j in range(i + 1, 2):
                other_var = abs(literal[j])
                matrix[var][other_var] -= 1
                matrix[other_var][var] -= 1
    return rank(matrix)

n = random.choice([5, 10, 15, 20, 30, 40])
k = min(2, n // 2)
cnf = generate_k_clique_cnf(n, k)
S_size = math.ceil(math.sqrt(n))
S = set(random.sample(range(1, n + 1), S_size))

r_S = matroid_rank_function(cnf, S)
min_elements = next(i for i in range(S_size, n + 1) if matroid_rank_function(cnf, set(range(1, i))))

conjecture_holds = min_elements == math.ceil(math.sqrt(n))
counterexample = "" if conjecture_holds else f"min_elements={min_elements}, expected={math.ceil}"

return {
    "metric_name": "Minimum elements to achieve rank",
    "metric_value": min_elements,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":

```

```

import sys
seeds = [int(arg) for arg in sys.argv[1:]] or generate_primes(30)

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[seeds.index(first_failing_seed)]['counterexample']}\")")
else:
    print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before producing data, preventing evaluation of support fraction or counterexamples. | next: Re-run test with detailed logging to identify crash cause and collect empirical evidence

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	87162
2	preregistration	ollama_remote	qwen3:8b	0	0	25390
3	novelty	ollama_remote	qwen3:8b	0	0	20538
4	novelty	ollama_remote	qwen3:8b	0	0	14875
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14238

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12441
7	judge	ollama_remote	qwen3:8b	0	0	15283

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 189928 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/b052cc185466.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/b052cc185466.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/b052cc185466.tar.gz (if generated)